

Development Plan

Software Engineering

Team 6, Blockchainerz

Robert Lam

Nathan Hum

Aidan Lao

Keira Laskoski

Table 1: Revision History

Date	Developer(s)	Change
Date1	Name(s)	Description of changes
Date2	Name(s)	Description of changes
...	...	...

[Put your introductory blurb here. Often the blurb is a brief roadmap of what is contained in the report. —SS]

[Additional information on the development plan can be found in the [lecture slides](#). —SS]

1 Confidential Information?

[State whether your project has confidential information from industry, or not. If there is confidential information, point to the agreement you have in place. —SS]

[For most teams this section will just state that there is no confidential information to protect. —SS]

2 IP to Protect

[State whether there is IP to protect. If there is, point to the agreement. All students who are working on a project that requires an IP agreement are also required to sign the “Intellectual Property Guide Acknowledgement.” —SS]

3 Copyright License

[What copyright license is your team adopting. Point to the license in your repo. —SS]

4 Team Meeting Plan

[This section is for meetings between the team members, and meetings between the team and their supervisor/advisor (as appropriate). —SS]

[How often will the team meet? where? when? —SS]

[If the meeting is a physical location (not virtual), out of an abundance of caution for safety reasons you shouldn’t put the location online —SS]

[To help schedule meetings, you can use the lecture/tutorial time when we do not have anything else scheduled. —SS]

[How often will you meet with your supervisor/advisor? when? where? —SS]

[Will meetings be virtual? At least some meetings should likely be in-person. —SS]

[How will the meetings be structured? There should be a chair for all meetings. There should be an agenda for all meetings. —SS]

5 Team Communication Plan

The team will use a Microsoft Teams group chat for regular communication. The team also has a shared Word document for collaborative editing of documents and planning during meetings. Github issues will be used to track each meeting and any associated agendas/minutes.

Communication with the supervisor will be done through email and Microsoft Teams. Every email with the supervisor will be sent to all team members. The team will meet regularly with the supervisor, subject to the supervisor's availability.

Github issues will be the primary method of communication for development tasks. Each issue will represent a development task and will be assigned to a team member; the team member will be responsible for updating the issue with their progress. The issues will be associated with a milestone representing the deliverable, and will be placed appropriately in the Kanban project according to its status (e.g., To Do, In Progress, Done). Feedback is expected from at least one other team member for each issue, through either a comment on the Github issue or reviewed in a meeting or group chat.

6 Team Member Roles

- **Agenda creator:** The agenda creator is responsible for creating the agenda for a meeting, and posting it to the meeting issue. The agenda creator will use the template for team meetings located in the repo, containing the agenda and an attendance tracker.
- **Notetaker:** The notetaker is responsible for taking notes during a meeting, and posting the meeting minutes to the meeting issue. The meeting minutes will be posted as a comment under the meeting issue. Multiple people may contribute to the notetaker role during a meeting, especially if there are multiple topics / individual updates from each person.
- **Pull request reviewer:** The pull request reviewer role is a role that is assigned to a team member for each pull request. The pull request reviewer is responsible for reviewing the code changes in the pull request, providing feedback, and approving or requesting changes to the pull request. Every team member is expected to review pull requests, but the pull request reviewer is the primary reviewer for that pull request. They are in charge of making the final decision on approving the request.
- **Developer:** The developer role is a role assigned to a team member for each development task. The developer is responsible for implementing the task, and updating the associated issue with their progress. Every team member is expected to take on this role and contribute to development

tasks as necessary, but the developer is the primary person responsible for completing a given task.

7 Workflow Plan

Regarding issues and deliverables: Each deliverable (and sub-deliverable if needed) will have a milestone associated with it. Each milestone will have a set of issues associated with it, representing the development tasks needed to complete the deliverable. These issues will be in the project Kanban board, and will be moved through the columns (To Do, In Progress, Done) as they are worked on.

Regarding pull requests: All code changes will be made in a new branch, and a pull request will be created once development is done for a particular task. See the branches and commit sections for naming convention. Typically, a branch will be created for each issue, but a branch may close several issues if they are related and can be completed together.

7.1 Branches & Commits

Branch naming convention: Each branch will be named according to the following convention: `<branch-type>/<branch-name>`. Like mentioned, development will be done in a new branch, and a pull request will be created once development is done for a particular task. The naming convention for branches will be kebab-case and will be based on the branch content as well as the branch type. For example, a branch for the issue "Implement user login" would be named "feature/implement-user-login". The current types of branches are feature and bugfix, but more types may be added as needed.

Commit naming convention: Each commit will be named according to the following convention: `<commit-type>: <commit-message>`. The naming convention for commits will be lowercase and will be based on the commit content as well as the commit type. For example, a commit for the issue "Implement user login" would be named "feat: implement user login". The current types of commits are feat, fix, docs, style, refactor, and test, but more types may be added as needed.

8 Project Decomposition and Scheduling

The project is organized around key milestones and overarching stages (refer to Table 2), progressing from initial project planning and requirements analysis through design, implementation, verification, and final demonstration. Each stage contains defined deliverables and deadlines to provide the team with a clear progression throughout the project. The schedule will remain flexible

where necessary, allowing the team to adjust tasks and responsibilities based on project needs, progress, and individual strengths.

Stage	Milestone	Deadline
Planning	Project selection	W2
Planning	Problem Statement + PoC + Development Plan + Team Charter	W4
Requirements	SRS + Hazard Analysis	W6
Verification	V&V Plan	W8
Design	Design Document	W10
Design	PoC Demo	W11
Implementation	Design Document Rev 0	W16
Implementation	Module Implementation	W16
Implementation	Rev 0 Demo	W18
Finalization	Final Demo + Documentation	W21
Finalization	V&V Report + CD	W22
Finalization	Final Demos	W23
Expo	Expo Materials	W25
Expo	Capstone EXPO	W26

Table 2: Project Milestones and Schedule

8.1 GitHub Project Usage

The project board will follow a Kanban-style workflow with columns for Backlog, Ready, In Progress, In Review, and Done. The project will be organized into milestones representing the overarching deliverables required throughout each stage of the project. For example, the initial stage contains milestones for the Project Statement and Development Plan, with individual tasks associated with each milestone. Each task will be represented as a card on the board and assigned to the appropriate team member. Cards will move through the Kanban columns as work progresses, allowing the team to track the status of individual tasks and overall milestone completion.

9 Proof of Concept Demonstration Plan

What is the main risk, or risks, for the success of your project? What will you demonstrate during your proof of concept demonstration to convince yourself that you will be able to overcome this risk?

10 Expected Technology

[What programming language or languages do you expect to use? What external libraries? What frameworks? What technologies. Are there major components

of the implementation that you expect you will implement, despite the existence of libraries that provide the required functionality. For projects with machine learning, will you use pre-trained models, or be training your own model? —SS]

[The implementation decisions can, and likely will, change over the course of the project. The initial documentation should be written in an abstract way; it should be agnostic of the implementation choices, unless the implementation choices are project constraints. However, recording our initial thoughts on implementation helps understand the challenge level and feasibility of a project. It may also help with early identification of areas where project members will need to augment their training. —SS]

Topics to discuss include the following:

- Specific programming language
- Specific libraries
- Pre-trained models
- Specific linter tool (if appropriate)
- Specific unit testing framework
- Investigation of code coverage measuring tools
- Specific plans for Continuous Integration (CI), or an explanation that CI is not being done
- Specific performance measuring tools (like Valgrind), if appropriate
- Tools you will likely be using?

[git, GitHub and GitHub projects should be part of your technology. —SS]

11 Use of AI and LLMs

[This section is a continuation of the previous section. The role of LLMs in your project as important enough to warrant their own section. How will your team be using LLMs for code and documentation development? What tools will you be using? How will you verify the output of your LLMs? —SS]

12 Coding Standard

The team will adhere to the coding standards outlined in Table 3.

Category	Details
Go Backend	Go code will follow the Effective Go guidelines and will be formatted using <code>gofmt</code> .
Rust Backend	Rust code will follow the official Rust API Guidelines and community style conventions, formatted using <code>rustfmt</code> and linted with <code>clippy</code> .
Rust Documentation	Rust functions, structs, and modules will use <code>rustdoc</code> doc comments (<code>///</code> and <code>//!</code>) following the conventions in the official Rust documentation guide .
Docker	Dockerfiles and container images will follow Docker's official best practices for maintainability, efficiency, and security.
Git	Git will be used for version control, with feature branches used to isolate development work.
Code Review	Changes will be submitted through GitHub pull requests and reviewed by another team member before being merged into the main branch. See the GitHub pull request review guidelines .

Table 3: Coding Standards

13 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

[This section is about the use of AI to write this document, not your planned use of AI for your project. You discuss that topic in one of the sections in the body of this report. —SS]

13.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

13.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

13.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

Appendix — Reflection

[Not required for CAS 741 —SS]

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. Why is it important to create a development plan prior to starting the project?
2. In your opinion, what are the advantages and disadvantages of using CI/CD?
3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Appendix — Team Charter

External Goals

For this Capstone, our team hopes to deliver a project that is both challenging and enjoyable to create. We aim to build something beyond simply fulfilling the requirements of a school project—something that we can be proud of and potentially carry forward into our professional careers. By focusing on producing a meaningful, well-developed, and high-quality project, we believe that a strong performance in the Capstone course will naturally follow.

Attendance

This section summarizes the ground rules and expectations regarding team member attendance.

Expectations

All team members are expected to attend weekly team meetings and meetings with our supervisor. Attendance at every Capstone course lecture is not required, provided that members remain up to date with the course material through either attending the lecture in person or reviewing the material independently.

Attendance at all required team and supervisor meetings will be tracked through GitHub Issues. All members are expected to maintain a 95% attendance rate throughout the project. When a meeting is held in person, team members should

make their best effort to attend in person. If a member is unable to attend in person, they should notify the team at least 5 hours in advance and attend virtually instead. If a member is unable to attend the meeting altogether, they must notify the team as soon as possible, preferably when they become aware of the conflict and at least 24 hours before the meeting when reasonably possible. Different expectations may apply depending on the circumstances.

Acceptable Excuse

Team members are expected to communicate with the rest of the team if they are unable to attend a meeting or meet a project deadline. Each member may miss up to three team meetings per term with prior notice. Acceptable reasons may include illness, family emergencies, religious observances, or previously arranged commitments that cannot reasonably be rescheduled.

If a member exceeds three absences in a term, the team will discuss the member's participation, expectations, and responsibilities. If the issue cannot be resolved

within the team, the team may involve the course professor or supervisor.

Team members are expected to meet all assigned deadlines. Exceptions will generally only be made in the event of an emergency, as outlined in the following section.

In Case of Emergency

If a team member is unable to meet a deadline or complete their assigned work due to an emergency, they must communicate this to the team as soon as they become aware that the work cannot be completed on time. The team will then

discuss appropriate accommodations and, if necessary, redistribute the affected work to ensure that project deadlines can still be met. If another team member must take on the incomplete work, the original team member will be expected to take on additional responsibilities during a subsequent phase, where appropriate, to help maintain a fair distribution of workload.

[borrows from [University of Portland Team Charter](#) —SS]

External Goals

[What are your team's external goals for this project? These are not the goals related to the functionality or quality of the project. These are the goals on what the team wishes to achieve with the project. Potential goals are to win a prize at the Capstone EXPO, or to have something to talk about in interviews, or to get an A+, etc. —SS]

Attendance

Expectations

[What are your team's expectations regarding meeting attendance (being on time, leaving early, missing meetings, etc.)? —SS]

Acceptable Excuse

[What constitutes an acceptable excuse for missing a meeting or a deadline? What types of excuses will not be considered acceptable? —SS]

In Case of Emergency

[What process will team members follow if they have an emergency and cannot attend a team meeting or complete their individual work promised for a team deliverable? —SS]

Accountability and Teamwork

Quality

[What are your team's expectations regarding the quality of team members' preparation for team meetings and the quality of the deliverables that members bring to the team? —SS]

Attitude

[What are your team's expectations regarding team members' ideas, interactions with the team, cooperation, attitudes, and anything else regarding team member contributions? Do you want to introduce a code of conduct? Do you want a conflict resolution plan? Can adopt existing codes of conduct. —SS]

Stay on Track

[What methods will be used to keep the team on track? How will your team ensure that members contribute as expected to the team and that the team performs as expected? How will your team reward members who do well and manage members whose performance is below expectations? What are the consequences for someone not contributing their fair share? —SS]

[You may wish to use the project management metrics collected for the TA and instructor for this. —SS]

[You can set target metrics for attendance, commits, etc. What are the consequences if someone doesn't hit their targets? Do they need to bring the coffee to the next team meeting? Does the team need to make an appointment with their TA, or the instructor? Are there incentives for reaching targets early? —SS]

Team Building

[How will you build team cohesion (fun time, group rituals, etc.)? —SS]

Decision Making

[How will you make decisions in your group? Consensus? Vote? How will you handle disagreements? —SS]